

2026.6.8

2026年6月8日 19:38

高精度算法

2026年7月7日 9:21

首先是通过string输入数据，转化成vector，此时需要倒序转化实现个位处于a[0]，因此vector转回string需要再倒序一遍

0、前置操作

0.1字符串转数组：记得通过字符'0'转换数据类型

```
vector<int> strto(string s) {  
    vector<int>n;  
    for(int i=s.length()-1;i>=0;i--) {  
        n.push_back(s[i]-'0');//将每一位转换成整形  
    }  
    return n;  
}
```

0.2数组转字符串（若最后直接倒序输出就不用这一步）

```
string tostr(vector<int>m) {  
    string n;  
    for(int i=m.size()-1;i>=0;i--) {  
        n+=(char)('0'+m[i]);  
    }  
    return n;  
}
```

1、高精度a+高精度b:

从第一位开始谁存在就加上，得到取余部分，部分除十，最后处理剩下部分给加上

```
vector<int> add(vector<int> m, vector<int> n) {  
    vector<int>t;  
    int temp=0;  
    for(int i=0;i<m.size()||i<n.size();i++) {  
        if(i<m.size()) temp+=m[i];  
        if(i<n.size()) temp+=n[i];  
        t.push_back(temp%10);  
        temp/=10;  
    }  
    while(temp>0) {  
        t.push_back(temp%10);  
        temp/=10;  
    }  
}
```

```

        return t;
    }
}

```

2、高精度a-高精度b:

2.1先要实现a>b

```

bool cmp(vector<int>m, vector<int>n) {
    if(m.size() != n.size())
        return m.size() > n.size();
    for(int i=m.size()-1; i>=0; i++) {
        if(m[i] != n[i]) return m[i] > n[i];
    }
    return true;
}

```

2.2借助borrow实现向高位借1的操作

2.3因为使用.push_back(), 所以最高位处于最后的位置, 进行去除前导零的操作

2.4因为是相减操作且每次重置temp, 所以最后不用再操作temp (temp是每一步的操作)

```

vector<int> sub(vector<int>m, vector<int>n) {
    vector<int>t;
    int borrow=0;
    for(int i=0; i<m.size(); i++) {
        int temp=m[i]-borrow;
        if(i<n.size()) temp-=n[i];
        borrow=0;
        if(temp<0) {
            temp+=10;
            borrow=1;
        }
        t.push_back(temp);
    }
    while(t.back()==0&& t.size()>1)
        t.pop_back();
    return t;
}

```

2.5主函数部分

```

cin>>a>>b;
vector<int>a1=strto(a);
vector<int>b1=strto(b);
if(cmp(a1, b1)) {
    cout<<tostr(sub(a1, b1));
}

```

```

    }
    else cout<<'-'<<tostr(sub(b1,a1));

```

3、高精度a*高精度b:

3.1使用了乘法竖式定理

3.2每一步分别处理当前和之前的res[i+j]

3.3去除前导零

3.4 *1LL避免指出int

```

vector<int>mul(vector<int>m, vector<int>n) {
    vector<int>res(m.size()+n.size(), 0);
    for(int i=0; i<m.size(); i++) {
        long long temp=0;
        for(int j=0; j<n.size() || temp; j++) {
            if(j<n.size())
                temp+=1LL*m[i]*n[j];
            temp+=res[i+j];
            res[i+j]=temp%10;
            temp/=10;
        }
    }
    while(res.back()==0&&res.size()>1) {
        res.pop_back();
    }
    return res;
}

```

3.5主函数部分

```

string a,b;
cin>>a>>b;
vector<int>a1=strto(a);
vector<int>b1=strto(b);
cout<<tostr(mul(a1,b1));

```

4、高精度a*低精度b:

4.1相比之下只需要单层循环

4.2需要处理剩下temp和前导零

4.3解决阶乘，使用双层循环和高精度加、乘

```

vector<int>mul_small(vector<int>m, int n) {
    vector<int>res;
    long long temp=0;

```

```

    for(int i=0;i<m.size();i++) {
        temp+=1LL*m[i]*n;
        res.push_back(temp%10);
        temp/=10;
    }
    while(temp>0) {
        res.push_back(temp%10);
        temp/=10;
    }
    while(res.back()==0&&res.size()>1)
        res.pop_back();
    return res;
}

```

4.4阶乘和

```

int s;
cin>>s;
string sum="0";
for(int i=1;i<=s;i++) {
    string t="1";
    for(int j=1;j<=i;j++) {
        //
sum=tostr(add(strto(sum),mul_small(strto(to_string(j)),i)));
        t=tostr(mul_small(strto(t),j));
    }
    sum=tostr(add(strto(sum),strto(t)));
}

```

5、高精度a/高精度b:

5.1返回商和余数

5.2先后各翻转一次

5.3去除前导零

5.4temp要使用ll

```

pair<vector<int>,int> div (vector<int>m,int n) {
    vector<int>res;
    long long temp=0;
    reverse(m.begin(),m.end());
    for(int i=0;i<m.size();i++) {
        temp=temp*10+m[i];
        res.push_back(temp/n);
        temp%=n;
    }
}

```

```

        reverse(res.begin(), res.end());
        while(res.back()==0&&res.size()>1) {
            res.pop_back();
        }
        return {res, temp};
    }
}

```

5.4主函数部分

```

string a;
int b;
cin>>a>>b;
vector<int>a1=strto(a);
pair<vector<int>, int>s;
s=div(a1, b);
cout<<tostr(s.first)<<' '<<s.second;

```

前缀和算法

2026年7月7日 9:21

0、补充定理

当两个数字 a , b 的差的余 m 时 (即 $(a-b)\%m==0$) , 此时 $a\%m==b\%m$

1、一维前缀和

```
for(int i=1;i<=n;i++){
    s[i]=s[i-1]+a[i];
}
```

2、二维前缀和

2.1从起点到 i , j 的和

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=m;j++){
        s[i][j]=arr[i][j]+s[i-1][j]+s[i][j-1]-s[i-1][j-1];
    }
}
```

2.2从 $x1,y1$ 到 $x2,y2$ 的和

```
int res=s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1];
```

2.3以边长判断时要满足两端都不超过范围

```
bool check(int h){
    for(int x1=1;x1+h-1<=n;x1++){
        int x2=x1+h-1;
        for(int y1=1;y1+h-1<=m;y1++){
            int y2=y1+h-1;
            int res=s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1];
            if(h*h==res)
                return true;
        }
    }
    return false;
}
```

2.4对上面换一种形式, 从正方形边长开始循环

```
for (int i=m;i<=5001;i++){
    for (int j=m;j<=5001;j++){
        int num=s[i][j]-s[i-m][j]-s[i][j-m]+s[i-m][j-m];
        maxn=max(maxn, num);
    }
}
```

3、差分数组

大体分为两种形式

3.1增量差分：首先对diff数组进行改变操作，再对diff进行一维分别行列求和操作，最后可加到原数组上

用处：区间批量加减

```
for(int i=1;i<=m;i++) {
    int x1,y1,x2,y2;
    cin>>x1>>y1>>x2>>y2;
    diff[x1][y1]++;
    diff[x2+1][y2+1]++;
    diff[x1][y2+1]--;
    diff[x2+1][y1]--;
}

for(int i=1;i<=n;i++) {
    for(int j=1;j<=n;j++) {
        diff[i][j]+=diff[i][j-1];
    }
}

for(int i=1;i<=n;i++) {
    for(int j=1;j<=n;j++) {
        diff[i][j]+=diff[i-1][j];
    }
}
```

3.2原生差值差分：首先通过后一项减去前一项得到diff数组，再进行操作

用处：算相邻差值、区间求和

深度优先搜索

2026年7月19日 18:43

0、考察题型

网格类 DFS（二维地图）、子集、组合问题（选数）、全排列、枚举所有方案

1、代码

```
int a[15];
bool vis[15];
void dfs(int step)
{
    if(step>n)
    {
        //输出一组排列
        return;
    }
    for(int i=1;i<=n;i++)
    {
        if(!vis[i])
        {
            vis[i]=true;
            a[step]=i;
            dfs(step+1);
            vis[i]=false;//回溯撤销
        }
    }
}
```

记忆化搜索

2026年8月5日 22:56

0、思路

斐波那契数列中左分支中求出来的添加到数组中充当已知，方便右分支

1、记忆化方法

本质为dfs+缓存答案

dfs中void改int，设立返回值

若算过直接返回值，不往下递归

2、代码

```
int dfs(int x,int y,int time)
{
    if(re[x][y][time]!=-1)
        return re[x][y][time];
    if(abs(x-r2)+abs(y-c2)>t-time)
        return re[x][y][time]=0;
    if(time>t)
        return re[x][y][time]=0;
    if(time==t)
    {
        if(x==r2&&y==c2)
            return re[x][y][time]=1;
        else return re[x][y][time]=0;
    }
    //01代表步数，到达边界条件的边界值
    int ans=0;
    for(int i=0;i<4;i++)
    {
        if(b[x+dx[i]][y+dy[i]]||x+dx[i]<1||x+dx[i]>n||y+dy[i]<1||y+dy[i]>m)
            continue;
        ans+=dfs(x+dx[i],y+dy[i],time+1);
    }
    return re[x][y][time]=ans;
}
```

广度优先搜索

2026年7月19日 18:43

0、考察题型

网格类，图论-无权图、最短路径

1、贪心思想

第一次访问到某一个状态时，一定是这个状态的最优情况。

2、代码

```
queue<poi>q;
q.push(poi(x,y));
st[x][y]=1;
while(!q.empty()){
    poi p=q.front();
    q.pop();
    int xa=p.xx,ya=p.yy;
    for(int i=1;i<=8;i++){
        int nx=xa+dx[i];
        int ny=ya+dy[i];
        if(nx>=1&&nx<=n&&ny>=1&&ny<=m){
            if(!st[nx][ny])
            {
                q.push(poi(nx,ny));
                arr[nx][ny]=arr[xa][ya]+1;
                st[nx][ny]=1;
            }
        }
    }
}
```

0、考察题型

网格最短路径

1、思路

1.1 $A^* = \text{bfs} + \text{启发式估价函数}$

1.2 $f = g + h$ 、

总优先级=实际代价+估价

1.3 曼哈顿距离 $h = |x_{\text{now}} - x_{\text{end}}| + |y_{\text{now}} - y_{\text{end}}|$

切比雪夫距离 $h = \max(|x_{\text{now}} - x_{\text{end}}|, |y_{\text{now}} - y_{\text{end}}|)$

1.4 使用优先队列，小根堆，每次弹出f最小的节点

2、代码

```

#include <bits/stdc++.h>
using namespace std;
struct Node{
    int f, g, h;
    int x, y;
    //小根堆：f小的优先出队
    bool operator>(const Node &b) const {
        return f > b.f;
    }
};
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};
int n, m;
int g[105][105]; //记录走到该点的真实最小代价
bool vis[105][105];
//曼哈顿估价
int geth(int x, int y, int ex, int ey) {
    return abs(x-ex) + abs(y-ey);
}

```

```

int main() {
    //优先队列小根堆
    priority_queue<Node, vector<Node>, greater<Node>>
q;

    //起点(sx, sy), 终点(ex, ey)
    int sx=1, sy=1, ex=n, ey=m;
    memset(g, 0x3f, sizeof g);
    //设定g数组为无穷大
    g[sx][sy]=0;
    int h0=geth(sx, sy, ex, ey);
    q.push({h0, 0, h0, sx, sy});
    while(!q.empty()) {
        auto cur=q.top(); q.pop();
        int x=cur.x, y=cur.y;
        if(x==ex&&y==ey) {
            cout<<cur.g<<endl;
            return 0;
        }
        if(vis[x][y]) continue;
        vis[x][y]=true;
        for(int i=0; i<4; i++) {
            int nx=x+dx[i];
            int ny=y+dy[i];
            //判断边界、障碍物省略
            int ng=cur.g+1;
            if(ng<g[nx][ny]) {
                //找到一条更近的路
                g[nx][ny]=ng;
                int h=geth(nx, ny, ex, ey);
                int f=ng+h;
                q.push({f, ng, h, nx, ny});
            }
        }
    }
}

```

```
    }  
    return 0;  
}
```

IDA*

2026年8月6日 22:31

搜索的剪枝

2026年8月6日 10:10

Dijkstra单源最短路

2026年8月5日 15:35

0、不能处理负权，负环

1、思路 $m \log n$

1.1建图，用vector<vector<edge>>邻接表存边

1.2初始化 dist 数组: dist [s]=0, 其余初始化为INT_MAX。

1.3 小根堆优先队列，存pair<距离，点号>，初始把{0, s}入堆

1.4主循环

取出堆顶，距离 d, 点 u; 如果 vis [u], 直接跳过。

标记 vis [u]=true, 代表 u 的最短距离已经确定

遍历 u 所有邻边edge(v, w):

- 重要: 如果 dist [u] 是 INT_MAX, 不能再做 dist [u]+w, 会 int 溢出!
- 如果dist[v] > dist[u] + w, **更新** dist [v], 把{dist[v], v}压入小根堆。
- 存入的dist代表最终结果, 会改变

2、特点

```
if(vis[u]) continue;
```

```
vis[u]=true; //永久锁死, 以后再也不处理u
```

3、代码

```
#include <bits/stdc++.h>
using namespace std;
int n, m, s;
struct edge{
    int t, we;
    edge(int to, int wei):t(to), we(wei) {}
};
int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>m>>s;
    vector<int>dist(n+1, INT_MAX); //最终距离
    vector<bool>vis (n+1, false); //代表最短路径是否确认
    vector<vector<edge>>a(n+1); //邻接表
    for(int i=0; i<m; i++) {
        int u, v, w;
        cin>>u>>v>>w;
        a[u].push_back(edge(v, w));
    }
}
```

```

}
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>q;
dist[s]=0;
q.push({0, s});
while(!q.empty()) {
    auto cur=q.top();
    q.pop();
    int d=cur.first, v=cur.second;
    if(vis[u])
        continue;
    vis[u]=true;
    for(auto e:a[u]) {
        int v=e.t;
        int w=e.we;
        if(dist[u]==INT_MAX)
            continue;
        if(dist[u]+w<dist[v]) {
            dist[v]=dist[u]+w;
            q.push({dist[v], v});
        }
    }
}
}
for(int i=1; i<=n; i++) {
    if(dist[i]==INT_MAX) {
        cout<<INT_MAX<<' ';
    }
    else
        cout<<dist[i]<<' ';
}
return 0;
}

```

SPFA

2026年8月21日 16:19

0、可以处理负权，可以检测负环

1、思路

1.1 时间复杂度：没负环 m ，有负环 $m*n$

1.2 SPFA 规则：只要某个点 v 的最短距离 $dist[v]$ 被**更新变小了**，那 v 的所有邻居，就有可能通过 v 得到更短的路径！所以我们要把 v 放到队列，接下来拿出来遍历它所有邻边，去松弛它的邻居。

1.3 dijk是永久不改变距离，spfa是**实时的距离,随时改变**

2、单遍代码

```
#include <bits/stdc++.h>
using namespace std;
const int INF = 0x3f3f3f3f;
//边结构体
struct Edge{
    int to, w;
    Edge(int t, int ww):to(t), w(ww) {}
};
int n, m;
vector<vector<Edge>> adj;
//单源SPFA, s起点, dist输出距离, false代表存在可达负环
bool spfa(int s, vector<int>& dist) {
    fill(dist.begin(), dist.end(), INF); //重置距离为无穷大
    vector<bool> inque(n+1, false); //标记是否在队列
    vector<int> cnt(n+1, 0); //入队次数, 判负环
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    inque[s] = true;
    cnt[s] = 1;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
```

```

    inque[u] = false; //出队取消标记
    for(auto& e : adj[u]) {
        int v = e.to, w = e.w;
        if(dist[u] != INF && dist[v] > dist[u] + w) {
            dist[v] = dist[u] + w;
            if(!inque[v]) { //不在队列才入队
                q.push(v);
                inque[v] = true;
                cnt[v]++;
                if(cnt[v] >= n) return false; //存在负环
            }
        }
    }
}

return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int start;
    cin >> n >> m >> start;
    adj.resize(n+1); //开辟邻接表
    //读入边
    for(int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].emplace_back(v, w);
    }
    vector<int> dist(n+1);
    bool ok = spfa(start, dist);
    if(!ok) {
        cout << "存在从起点可达的负环\n";
    } else {
        //输出起点到各个点距离
        for(int i = 1; i <= n; i++) {
            if(dist[i] == INF) cout << "INF ";
            else cout << dist[i] << " ";
        }
    }
}

```

```

    }
}
return 0;
}

```

3、多遍spfa

```

#include <bits/stdc++.h>
using namespace std;
const int INF = 0x3f3f3f3f;
// 边结构体
struct Edge{
    int to, w;
    Edge(int t, int ww):to(t), w(ww) {}
};
int n, m;
vector<vector<Edge>> adj;    // 邻接表
vector<vector<int>> dist_all; // 全源最短路矩阵 dist_all[s]
                             [t] s到t
// spfa单源最短路, s起点, dist存输出结果, 返回false代表存在
可达负环
bool spfa(int s, vector<int>& dist) {
    fill(dist.begin(), dist.end(), INF); // 重置距离为无穷大
    vector<bool> inque(n+1, false);      // 标记点是否在队列中
    vector<int> cnt(n+1, 0);              // 记录入队次数, 判负
    环
    queue<int> q;
    dist[s]=0;
    q.push(s);
    inque[s]=true;
    cnt[s]=1;
    while(!q.empty()) {
        int u=q.front();q.pop();
        inque[u]=false; // 出队, 取消队列标记
        for(auto& e:adj[u]) {

```

```

int v=e.to,w=e.w;
if(dist[u]!=INF&&dist[v]>dist[u]+w){
    dist[v]=dist[u]+w;
    if(!inque[v]){ // 不在队列才入队
        q.push(v);
        inque[v]=true;
        cnt[v]++;
        if(cnt[v]>=n) return false; // 入队>=n

```

次, 存在负环

```

    }
    }
    }
    }
    return true;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin>>n>>m;
    adj.resize(n+1); // 开辟邻接表空间,
    // 下标1~n
    dist_all.resize(n+1,vector<int>(n+1)); // 开辟全源矩阵空
    // 间
    // 读入边
    for(int i=0;i<m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        adj[u].emplace_back(v,w);
    }
    vector<int> dist(n+1);
    // n遍spfa, 每个点依次做源点
    for(int s=1;s<=n;s++){
        spfa(s,dist);
        for(int t=1;t<=n;t++) dist_all[s][t]=dist[t];
    }
    // 输出全源最短路矩阵

```

```
for(int i=1;i<=n;i++) {  
    for(int j=1;j<=n;j++) {  
        if(dist_all[i][j]==INF) cout<<"INF ";  
        else cout<<dist_all[i][j]<<" ";  
    }  
    cout<<"\n";  
}  
return 0;  
}
```

Kruskal

2026年8月21日 20:18

1、思路:

使用并查集、排序边

2、代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;
const int N=200005;
int n,m;
int ans,k;
struct Edge
{
    int u,v,w;
}edge[N];
int fa[N],si[N];
int find(int x){
    if(fa[x]==x)
        return x;
    return fa[x]=find(fa[x]);
}
bool cmp(const Edge &x,const Edge &y){
    return x.w<y.w;
}
void kruskal(){
    ans=0;
    k=0;
    for(int i=1;i<=n;i++){
        fa[i]=i;
        si[i]=1;
    }
```

```

for(int i=1;i<=m;i++) {
    int x=find(edge[i].u);
    int y=find(edge[i].v);
    if(x!=y) {
        ans+=edge[i].w;
        if(si[x]<si[y])
            swap(x,y);
        fa[y]=x;
        si[x]+=si[y];
        k++;
        if(k==n-1)
            break;
    }
}
if(k<n-1) {
    cout<<"orz";
    exit(0);
}
}

signed main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>m;
    for(int i=1;i<=m;i++) {
        cin>>edge[i].u>>edge[i].v>>edge[i].w;
    }
    sort(edge+1, edge+m+1, cmp);
    kruskal();
    cout<<ans;
    return 0;
}

```

Prim

2026年10月2日 18:01

1、思路：

每次循环取距离起点部分最近的点，加入此集合，再用此点遍历其相邻的点更新距离

2、例题代码

朴素 $n*n$ ，平面欧几里得最小生成树

```
#include <bits/stdc++.h>
using namespace std;
const int N=5005;
int n;
double ans=0;
double x[N],y[N],dis[N];
bool vis[N];
double len(double xa,double ya,double xb,double yb){
    return sqrt((xa-xb)*(xa-xb)+(ya-yb)*(ya-yb));
}
int main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>x[i]>>y[i];
        dis[i]=1e9;
    }
    vis[1]=true;
    dis[1]=0;
    for(int i=1;i<=n;i++){
        double minn=1e9;
        int p=1;
        for(int j=1;j<=n;j++){
            if(!vis[j]&&dis[j]<minn){
```

```

        p=j;
        minn=dis[j];
    }
}
vis[p]=true;
ans+=dis[p];
for(int j=1;j<=n;j++) {
    dis[j]=min(dis[j],len(x[p],y[p],x[j],y[j]
));
}
}
cout<<fixed<<setprecision(2)<<ans<<' \n' ;
return 0;
}

```

贪心思想

2026年7月20日 10:47

- 1、字典序最小从后方选取元素修改构造
- 2、搜索时第一次访问到某结点时为最优状态

stl的使用

2026年8月5日 20:01

1、vector

```
vector<int> v; //空vector, 存int
vector<int> v(10); //10个元素, 默认初始0
vector<int> v(10, 5); //10个元素, 全部初始化为5
vector<int> v2 = {1, 2, 3}; //直接初始化
//邻接表写法: 数组, 每个元素是vector
vector<int> adj[100010];
```

大小根堆

2026年9月22日 15:17

```
// 默认：大根堆
priority_queue<int> q;
// 小根堆写法
priority_queue<int, vector<int>, greater<int>>> q;
// 小根堆，用于Dijkstra
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>> q;
```

最大子数组

2026年8月21日 16:20

0、动态规划前提

- 1、最优子结构
- 2、重叠子问题
- 3、分治算法是自上而下的递归，迭代dp是自下而上的递归

1、思路

dp[i]: 以第i个元素为结尾的最大连续子数和组

状态转移方程: $dp[i] = \max(nums[i], dp[i-1] + nums[i]);$

: 看当dp[i-1]小于0的时候为nums[i]

2、代码

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int maxSubArray(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n);
    dp[0] = nums[0];
    int ans = dp[0];
    for(int i = 1; i < n; ++i) {
        dp[i] = max(nums[i], dp[i-1] + nums[i]);
        ans = max(ans, dp[i]);
    }
    return ans;
}
int main() {
    vector<int> arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    cout << maxSubArray(arr) << endl; //输出6, 对应子数组[4, -1, 2, 1]
    return 0;
}
```

01背包问题

2026年9月8日 14:26

1、思路

由二维dp可知一维dp中 i 件物品取决于 i-1 件物品，所以只保留前一次的数据

状态转移方程： $dp[j] = \max(dp[j], dp[j - w[i]] + v[i]);$

容量从后往前遍历，可以保证dp[j - w[i]]还是上一层 i-1得到的

2、代码

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main()
{
    int n, V;
    cin >> n >> V;
    vector<int> w(n+1), v(n+1);
    for(int i = 1; i <= n; i++)
    {
        cin >> w[i] >> v[i];
    }
    vector<int> dp(V+1, 0);
    for(int i = 1; i <= n; ++i)
    {
        // j倒着遍历，避免重复选同一个物品，否则成为完全背包
        for(int j = V; j >= w[i]; --j)
        {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    cout << dp[V] << endl;
    return 0;
}
```

旅行商问题

2026年9月8日 14:26

最长公共子序列

2026年9月8日 14:27

状态压缩

2026年9月8日 14:27

进入操作

2026年8月28日 8:05

1、客户端连接

调用cmd输入mysql -u root -p

再输入密码123456

(-u是调用用户名 -p是调用密码)

exit 退出该用户

2、启动与关闭sql

net start mysql80

net stop mysql80

(需要以管理员身份运行)

DDL

2026年8月28日 8:06

一、数据库

1、创建数据库

create database [if not exists] itcast;

2、删除数据库

drop database [if exists] itcast;

3、选择数据库

use itcast;

4、展示所有数据库

show databases;

5、查看当前选择的数据库

select database();

二、基本表

1、创建基本表

```
create table tbuser(  
    -> id int comment '编号',  
    -> name varchar(50) comment '姓名',  
    -> age int comment '年龄',  
    -> gender varchar(1) comment '性别'  
    -> )comment '用户表';
```

中间使用逗号分隔开，comment后面的注释一般加单引号

2、查询基本表

show tables;查询当前数据库所有表

desc tbuser;查询表结构

show create table tbuser;查询指定表的建表语句

3、表操作

alter table tbuser **add** nickname varchar(20);

: 给表tbuser**添加**类型长度为varchar(20)的nickname字段

alter table tbuser **modify** name varchar(30);

:把字段name的数据类型**修改**为varchar(30)

alter table tbuser **change** age ageee int comment '年龄';

: 把字段age**修改**为ageee,类型为int

alter table tbuser **drop** nickname;

: **删除**字段nickname

4、修改表名

alter table tbuser **rename** to tbuserrr;

5、删除表

drop table if exists tbuser;

:删除表tbuser

```
truncate table tbuser;
```

:删除并重新创建表tbuser;

数据类型

2026年8月28日 18:55

一、整型数据

tinyint	1字节	-128~127	0~255
smallint	2字节	-32768~32767	0~65535
mediumint	3	-8388608 ~ 8388607	
int/ integer	4	$-2^{31} \sim 2^{31}-1$	$0 \sim 2^{32}-1$
bigint	8	$-2^{63} \sim 2^{63}-1$	

unsigned

二、浮点数，定点数

float	4
double	8

decimal (M, D) 定点数，存金额必用
一共M位，小数占2位

三、字符串类型

char (n)	定长，n最大为255，一定会占用n个字符的内存
varchar (n)	变长
text	文本
tinytext/ mediumtext/ longtext	

n存的是字符不是字节，utfmb4里面一个汉字占三个字节

四、日期时间类型

date	3	yyyy-mm-dd
time	3	hh: mm: ss
datetime	8	yyyy-mm-dd hh: mm: ss
year	1	yyyy

五、二进制，存字节流

tinyblob	
blob	
mediumblob	
longblob	

六、注释形式

单行注释：1、双横杠+空格 “-- 注释部分”

2、#注释部分

多行注释：/*注释

部分*/

DML

2026年8月29日 8:31

一、添加数据

1、给指定字段添加数据

```
insert into tbuser (id, name) values(1, 'xiaoli');
```

: 未添加的为null

2、给全部字段添加数据

```
insert into tbuser (id, name, ageee, gender) values(1, 'xiaoli', 18, '男');
```

3、批量添加数据

```
insert into tbuser (id, name) values(1, 'xiaoli'), (2, 'xiaoli');
```

```
insert into tbuser (id, name, ageee, gender) values(1, 'xiaoli', 18, '男'),  
(2, 'xiaoli', 28, '男');
```

二、修改数据

4、修改数据

```
update tbuser set name='wenwen', gender='女' where id=2;
```

: where语句可以没有, 即修改全部数据

三、删除数据

5、删除数据

```
delete from tbuser where id=2;
```

: where语句可以没有, 即删除全部数据

DQL

2026年8月29日 11:51

一、基础查询

1、查询指定字段

```
select name,gender from tbuser;
```

2、查询所有字段返回

```
select * from tbuser;
```

：少用不直观效率低

3、查询所有员工的性别，起别名

```
select gender as '性别' from tbuser;
```

```
select gender '性别' from tbuser;
```

：as可以省略，起别名之后只能使用别名，原名不能用

4、查询员工的性别，不重复

```
select distinct gender from tbuser;
```

二、条件查询 where

1、在任意后面加上where语句就可

：判空is null,判非空is not null

：and or not

：between ... and ...

：in(...)包含其一

：name like '_ _';判断name是为2个字符，_指一个字符

：name like '%X';判断最后一位是X的name，%指多个字符

三、聚合函数

1、count () 统计数量

2、max () 最大值

3、min () 最小值

4、avg () 平均值

5、sum () 求和

在括号里面填入字段名，*可代之表的所有数据（通配符）

where后面不可以使用聚合函数

```
select avg(ageee) from tbuser where gender='男';
```

四、分组查询 group by

select 字段列表 重命名 from 表名 【where 条件】 group by 分组字段名 【having 分组后过滤条件】

```
select gender, sum(ageee) agesum from tbuser where id=1 group by gender having agesum>2;
```

分组之后，查询的字段一般为聚合函数和分组字段，查询其他字段无任何意义

执行顺序where>聚合函数>having

五、排序查询 order by

```
select * from tbuser order by id asc, ageeee desc;
```

：先按照id升序排序，再按照ageeee降序排序

升序 asc，可以省略

降序 desc

六、分页查询 limit

select 字段列表 from 表名 limit 起始索引, 查询记录数;

起始索引= (查询页码-1) * 每页显示记录数

如果查询的是第一页，起始索引可以忽略，直接简写为limit 5

```
select *from tbuser limit 0,5;
```

：查询第一页，5个记录

```
select *from tbuser limit 5,5;
```

：查询第二页，5个记录

DCL

2026年8月31日 10:09

一、用户管理

1、创建用户heima, 只能在当前主机访问

```
create user 'heima'@'localhost' identified by '123456';
```

'localhost' 代表自己主机

'%' 代表任意主机

2、修改用户密码

```
alter user 'heima'@'localhost' identified with mysql_native_password  
by '1234';
```

3、删除用户

```
drop user 'heima'@'localhost';
```

二、权限控制

1、查询权限

show grants for '用户名'@'主机名';

2、授予权限

grant 权限列表 on 数据库名.表名 to '用户名'@'主机名';

3、撤销权限

revoke 权限列表 on 数据库名.表名 from '用户名'@'主机名';

注：多个权限之间使用逗号分隔，数据库名和表名可以使用*进行通配，
*代表所有,不能用来代指全部权限

4、权限列表

all、all privileges	所有权限
select	查询数据
insert	插入数据
update	修改数据
delete	删除数据
alter	修改表
drop	删除数据库/ 表/ 视图
create	创建数据库/ 表

字符串函数

2026年8月28日 8:06

concat(S1,S2...Sn)	字符串拼接（只要一个为null，就返回null
lower(str)	将字符串str全部转为小写
upper(str)	将字符串全部转为大写
lpad(str, n, pad)	用字符串pad对str的左边进行填充，达到n个字符串长度
rpadd(str, n, pad)	用字符串pad对str的右边进行填充，达到n个字符串长度
trim(str)	去掉字符串头部和尾部的空格
substring(str, start, len)	返回从字符串str从start位置起的len个长度的字符串

数值函数

2026年8月31日

15:43

ceil (x)	向上取整
floor (x)	向下取整
mod (x, y)	返回x/ y的模
rand ()	返回0~1内的随机数
round (x, y)	求参数x的四舍五入的值，保留y位小数

日期函数

2026年8月31日 15:54

curdate ()	返回当前日期
curtime ()	返回当前时间
now ()	返回当前日期和时间
year (date)	获取指定给你date的年份
month (date)	获取指定date的月份
day (date)	获取指定date的日期
date_add (date, interval expr type)	返回一个日期/ 时间值加上一个时间间隔expr后的时 间值
datediff (date1, date2)	返回起始时间date1和结束时间date2之间的天数

```
select date_add(now() , interval 30 day );
select datediff(' 2026-08-31' , ' 2006-12-12' ); 第一个时间减去第二个时间
```

流程函数

2026年8月31日 17:41

if (value, t, f)	如果value为true，则返回t，否则返回f
ifnull (value1, value2)	如果value1不为空，返回value1，否则返回value2
case when [val1] then [res1] ... else [default] end	如果val1的true，返回res1， ... 否则返回default默认值
case [expr] when [val1] then [res1] ... else [default] end	如果expr的值等于val1，返回res1， ... 否则返回default默认值

```
select
  name,
  case address when '北京' then '一线城市' when '上海' then '一线城市' else
'二线城市' end
from tbuser;
```

约束分类

2026年8月31日 21:51

约束	描述	关键字
非空约束	限制该字段的数据不能为null	not null
唯一约束	保证该字段的所有数据都是唯一、不重复的	unique
主键约束	主键是一行数据的唯一标识，要求非空且唯一	primary key
默认约束	保存数据是，如果未指定该字段的值，则采用默认值	default
检查约束	保证字段值满足某一个条件	check
外键约束	用来让两张表的数据之间建立连接，保证数据的一致性和完整性	foreign key

注：约束是作用于表中字段上的，可以在创建表/修改表的时候添加约束（注释前）

自增：auto_increment，下面给id自增则在上传数据时就不用上传id

```
create table ituser(  
    id int primary key auto_increment,  
    name varchar(10) not null unique ,  
    age int check ( age>0 and age<=120 ),  
    status char(1) default '1',  
    gender char(1)  
)comment '第二用户表';
```

```
create table ituser(  
    id int auto_increment,  
    name varchar(10) not null unique ,  
    age int check ( age>0 and age<=120 ),  
    status char(1) default '1',  
    gender char(1),  
    primary key(id)
```

```
) comment '第二用户表';
```

外键约束

2026年9月1日 9:11

一、添加外键

1、

```
alter table tbuser add constraint id_age foreign key (ageee) references ituser(id);
```

alter table 表名 add constraint 外键名称 foreign key (外键字段名) references 主表 (主表列名)

是在tbuser表里添加一个外键连接到ituser的主键，也就是主表列名必须为主键

2、

```
create table 表名 (  
    字段名 数据类型,  
    ...  
    [constraint] [外键名称] foreign key (外键字段名) references 主表 (主表列名)  
);
```

二、删除外键

alter table 表名 drop foreign key 外键名称;

三、外键删除/更新行为

no action	当在父表中删除/更新对应记录时，首先检查该记录是否有对应外键，如果有则不允许删除
restrict	同上
cascade	父表删除/更新时，首先检查该记录是否有对应外键，如果有则也删除/更新外键在子表中的记录
set null	父表删除对应记录时，首先检查外键，如果有则设置子表中该外键值为null（要求允许取null）
set default	父表有变更时，子表将外键列设置成一个默认的值（版本不支持）

格式：

在添加外键后面加上on update cascade on delete cascade;

```
alter table tbuser add constraint id_age foreign key (ageee) references ituser(id) on update cascade on delete cascade;
```

多表关系

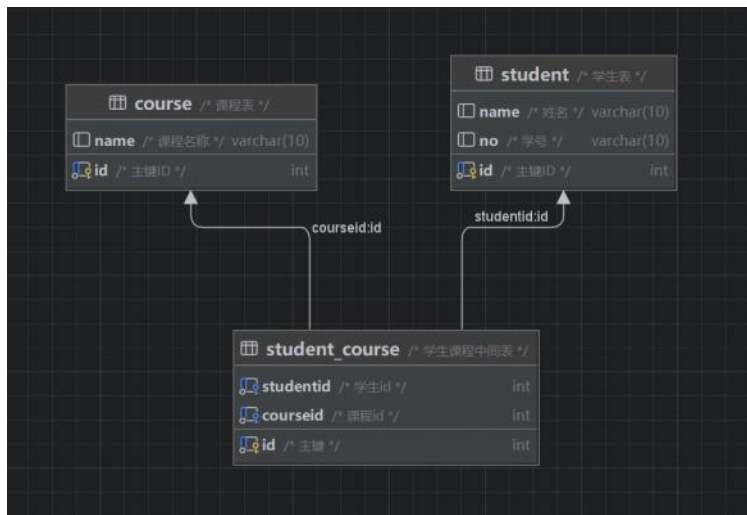
2026年9月1日 15:56

一、表关系

一对多：

创建一个中间表用来连接几个表

```
create table student(  
    id int auto_increment primary key comment '主键ID',  
    name varchar(10) comment '姓名',  
    no varchar(10) comment '学号'  
)comment '学生表';  
  
insert into student (name, no) values ('黛绮丝','2000100101'),('谢  
逊','2000010102'),('殷天正','2000010103'),('韦一笑','2000010104');  
  
create table course(  
    id int auto_increment primary key comment '主键ID',  
    name varchar(10) comment '课程名称'  
)comment '课程表';  
  
insert into course values  
(null,'java'),(null,'mysql'),(null,'hadoop');  
insert into course values (null,'php');  
  
create table student_course(  
    id int auto_increment primary key comment '主键',  
    studentid int not null comment '学生id',  
    courseid int not null comment '课程id',  
    constraint fk_stuid foreign key (studentid) references  
student(id),  
    constraint fk_coid foreign key (courseid) references course(id)  
)comment '学生课程中间表';  
  
insert into student_course values  
(null,1,1),(null,1,2),(null,1,3),(null,2,2),  
                                (null,2,3),(null,3,4);
```



一对一：

新建一个字段用来代指外键，且设置为唯一的

二、笛卡尔积

1、会出现emp和dept数量的乘积种情况

```
select *  
from emp, dept;
```

2、不会出现笛卡尔积这种情况

```
select *  
from emp, dept where emp.dept_id=dept.id;
```

连接查询

2026年9月1日 18:22

一、内连接

内连接查询的是两张表交集的地方

1、隐式内连接

```
select e.name, d.name from emp e, dept d where d.id=e.id;
```

(where管筛选条件)

2、显式内连接

select 字段列表 from 表1 [inner] join 表2 on 连接条件

```
select e.name, d.name  
from emp e inner join dept d on e.dept_id = d.id;
```

其中inner可以省略

(on管关联条件)

二、外连接

```
select d.*, e.*  
from dept d left outer join emp e on e.dept_id = d.id;
```

left为左外连接，right为右外连接

左外连接包含左表所有数据和表1表2交集部分的数据

其中outer可以省略

三、自连接

：对一个表起两个别名再使用内连接和外连接

```
select a.name, b.name  
from emp a, emp b where a.managerid=b.id;
```

```
select a.name 员工, b.name 领导
```

```
from emp a left join emp b on a.managerid=b.id;
```

联合查询

2026年9月1日 21:52

```
select * from emp where salary<5000  
union all  
select * from emp where age>50;
```

- 1、对于union查询，就是把多次查询的结果合并起来，形成一个新的查询结果集
- 2、对于联合查找的多张表的列数必须保持一致，字段类型也需要保持一致
- 3、union all会将所有数据直接并在一起，union会去重

子查询

2026年9月1日 18:22

1、标量子查询

`select name,dept_id from emp where dept_id=(select id from dept where name='销售部');`

2、列子查询

：子查询返回的结果是一列（可以是多行）

in	在指定的集合范围之内
not in	不在指定的集合范围之内
any	子查询返回列表中，有任意一个满足即可
some	与any等同
all	子查询返回列表的所有值都必须满足

`select * from emp where salary >all(select salary from emp where dept_id=(select id from dept where name='市场部'));`

查询出来比市场部所有人工资都高的员工

其中子查询市场部员工的工资是一列

`select *from emp where dept_id in(select id from dept where name='研发部' or name='市场部')`

查询研发部和市场部所有员工信息

3、行子查询

：子查询返回的结果是一行（可以是多列）

常用操作符：=、<>、in、not in

<>为不等于

`select *from emp where (salary,managerid)=(select salary,managerid from emp where name='张无忌');`

采用（, , ）=（子查询）的形式

4、表子查询

：子查询返回的结果是表，以表的形式查询

常用操作符：in

`select * from emp where (job,salary) in (select job,salary from emp where name='鹿杖客' or name='宋远桥');`

查询与鹿杖客或宋远桥职位薪资相同的人的信息

`select *from (select * from emp where entrydate > '2006-01-01') e left join dept d on e.dept_id=d.id;`

将查询到的结果当成一个表，可以起别名

事务操作

2026年9月2日 17:41

：事务是一组操作的集合，要么全部执行成功，要么全部执行失败

1、查看/设置事务提交方式

```
select @@autocommit;  
set @@autocommit=0;
```

默认提交方式是1，自动提交

：当提交方式是1时，在事务未遇到错误时会依次实时进行操作

2、提交事务

：当提交方式时0时，每次事务操作完成之后需要输入commit；进行提交
commit；

3、回滚事务

rollback；

4、开启事务

start transaction 或 begin；

ACID

2026年9月2日 21:43

A原子性：事务时不可分割的最小操作单元，要么全部成功，要么全部失败

C一致性：事务完成时，必须使所有的数据都保持一致状态

I 隔离性：数据库系统提供的隔离机制，保证事务在不受外部并发操作影响的独立环境下运行

D持久性：事务一旦提交或回滚，它对数据库中的数据的改变就是永久的

并发事务问题

2026年9月2日 21:56

一、事务问题

笼统的讲是两个事务对同一个数据库操作时产生的错误

脏读：一个事务读到另外一个事务还没有提交的数据

不可重复读：一个事务先后读取同一条数据，但两次读取的数据不同

幻读：一个事务按照条件查询数据时，没有对应的数据行，但是在插入数据时，又发现这行数据已经存在，好像出现了一个幻影

二、事务隔离级别

隔离级别越高，数据越安全，效率越低

隔离级别	脏读	不可重复读	幻读
read uncommitted	会	会	会
read committed	不会	会	会
repeatable read (默认)	不会	不会	会
serializable	不会	不会	不会

1、查看事务隔离级别

```
select @@transaction_isolation;
```

2、设计事务隔离级别

```
set [ session / global ] transaction isolation level [ 隔离级别 ];
```

session指只在当前这个数据库连接生效

global指对所有新建立的连接都生效

体系结构

2026年9月3日 17:28

一、mysql的体系结构

- 1、连接层
- 2、服务层
- 3、引擎层
- 4、存储层

二、存储引擎选择查看

1、查看支持的引擎

```
show engines;
```

2、通过查询建表结构查看信息

```
show create table dept;
```

3、创建表指定引擎

```
create table my_myisam(  
    id int,  
    name varchar(10)  
)engine = myisam;
```

默认引擎为innodb

引擎特点

2026年9月3日 18:03

一、innodb

1、特点

- DML操作遵循ACID模型，支持事务；

- 行级锁，提高并发访问性能

- 支持外键 FOREIGN KEY约束，保证数据的完整性和正确性

2、文件

xxx.ibd: xxx代表的是表名，innodb引擎的每张表都会对应这样一个表空间文件，存储该表的表结构、数据、和索引

参数: innodb_file_per_table

索引结构

2026年9月10日 10:08

- 1、mysql的索引是在存储引擎层实现的
- 2、B+树是在B树的扩展
- 3、B树在所有结点都存储键值和数据，B+树只在叶子结点存储键值和数据
- 4、索引没有特别指明的话就指B+树结构组织的索引

索引		InnoDB	MyISAM	Memory
B+树索引	最常见的索引类型，大部分引擎都支持B+树索引	支持	支持	支持
Hash索引	底层数据节后是层哈希表实现的，只有精确匹配索引列的查询才有效， 不支持范围查询	不	不	支持
R-tree(空间索引)	特殊索引类型，主要用于地理空间数据类型	不	支持	不
full-text(全文索引)	是一种通过进阿里倒排索引，快速匹配文档的方式	5.6之后支持	支持	不

- 5、B树：多路平衡查找树，当树的度为5时，如果节点包含了5个元素，则将数值中间的元素向上成为结点，左孩子为比中间小的元素，右孩子为比中间大的元素
- 6、B+树：所有数据都会出现在叶子结点，叶子结点形成一个单向列表，mysql又对其优化使变成双向链表
- 7、hash：通过一定的hash算法，将键值换算成hash值，存储在hash表中
如果两个hash相同了，就通过链表存储在后面
不能范围查询，不能利用索引排序，查询效率高

索引分类

2026年9月10日 10:35

分类	含义	特点	关键字
主键索引	针对表中主键创建的索引	默认自动创建，只能有一个	primary
唯一索引	避免同一个表中某数据列中的值重复	可以有多个	unique
常规索引	快速定位特定数据	可以有多个	
全文索引	查找的是文本中的关键字，而不是比较索引中的值	可以有多个	fulltext

根据innodb当中索引的存储形式

分类	含义	特点
聚焦索引	将数据存储与索引放到了一块，索引结构的叶子节点保存了行数据	必须有而且只有一个
二级索引	将数据和索引分开存储，索引结构的叶子结点关联的是对应的主键	可以存在多个

- 1、聚焦索引选取规则：

如果存在主键，主键索引就是聚集索引

如果不存在主键，将使用第一个唯一索引作为聚集索引

如果表没有主键，或没有合适的唯一索引，则innodb会自动生成一个rowid作为隐藏的聚集索引
- 2、回表查询
- 如果在二级索引里面存name，那么查询 where name = 'arm' ，
- 会首先先走二级索引通过name查到id，再再聚集索引通过id查到行数据

索引语法

2026年9月10日 11:00

1、创建索引

```
create index index_name on tbuser (name, gender);
```

```
create unique index index_name on tbuser (name, gender);
```

```
create fulltext index index_name on tbuser (name, gender);
```

选择多个字段就是联合索引

2、查看索引

```
show index from tbuser;
```

3、删除索引

```
drop index index_name on tbuser;
```

三种实现方式

2026年9月22日 9:08

1、继承 Thread类 实现

```
public class MyThread extends Thread{
    @Override
    public void run() {
        String name = getName(); //是thread类中的方法
        for (int i = 0; i < 100; i++) {
            System.out.println(name+"Hello,World");
        }
    }
}
```

```
public class Demo {
    public static void main(String[] args) {
        MyThread mt1 = new MyThread();
        MyThread mt2 = new MyThread();
        mt1.setName("线程1"); //是thread类中的方法
        mt2.setName("线程2");
        mt1.start();
        mt2.start();
    }
}
```

2、实现 Runnable接口

·没有继承Thread，所以需要新建Thread来使用方法

步骤：类实现Runnable，重写run()方法，交给Thread构造器，调用start()

```
public class MyRunnable implements Runnable{
    @Override
    public void run() {
        Thread t = Thread.currentThread();
        for (int i = 0; i < 100; i++) {
            System.out.println(t.getName()+"Hello,World");
        }
    }
}
```

```
public class Demo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
    }
}
```

```

        Thread t2 = new Thread(new MyRunnable());
        t1.setName("1");
        t2.setName("2");
        t1.start();
        t2.start();
    }
}

```

3、实现 Callable接口 和 FutureTask

步骤：实现Callable<V>，重写 call()（有返回值，抛异常），用FutureTask包装，再传入Thread

```

import java.util.concurrent.Callable;

public class MyCallable implements Callable<Integer> {

    @Override
    public Integer call() throws Exception {
        return 100;
    }
}

import java.util.concurrent.ExecutionException;
import java.util.concurrent.Future;
import java.util.concurrent.FutureTask;

public class Demo3 {
    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
        FutureTask<Integer> ft = new FutureTask<>(new MyCallable());
        Thread t1 = new Thread(ft);
        t1.start();
        Integer res = ft.get();
        System.out.println(res);
    }
}

```

常见的成员方法

2026年9月22日 10:39

1	String getName()	返回此线程名称
2	void setName(String name)	设置线程的名字（构造方法也可以设置名字）
3	<i>static</i> Thread currentThread()	获取当前线程的对象
4	<i>static</i> void sleep(long time)	让线程休眠指定实践，毫秒
5	setPriority(int newPriority)	设置线程的优先级
6	final int getPriority()	获取线程的优先级
7	final void setDaemo(boolean on)	设置为守护线程
8	public static void yield()	出让线程/礼让线程
9	public static void join()	插入线程/插队线程

2、通过构造方法来设置名字

2.1

需要在MyThread里面alt+insert设置无参和string构造器

```
MyThread mt1 = new MyThread("名字1");
```

```
MyThread mt2 = new MyThread();
```

2.2

在Thread类里面命名不需要在定义构造器

```
Thread t1 = new Thread(new MyRunnable(),"1");
```

2.3

若什么也没设置，则为默认值Thread-X（X为序号从0开始）

3、main为主线程，静态方法直接使用类名调用

4、

```
System.out.println("eeeeee");
```

```
Thread.sleep(5000);
```

```
System.out.println("12344");
```

会间隔5秒输出，时间的底层为long，（int只能存24天）

在子类中使用sleep跑出异常时只能使用try-catch

，因为父类的run方法没有抛出异常，子类就不能抛出异常

5、设置优先级最高为10，最低为1，默认为5

优先级只能代表优先的概率，并不是一定优先

```
MyThread mt1 = new MyThread("名字1");  
mt1.setPriority(9);  
System.out.println(mt1.getPriority());
```

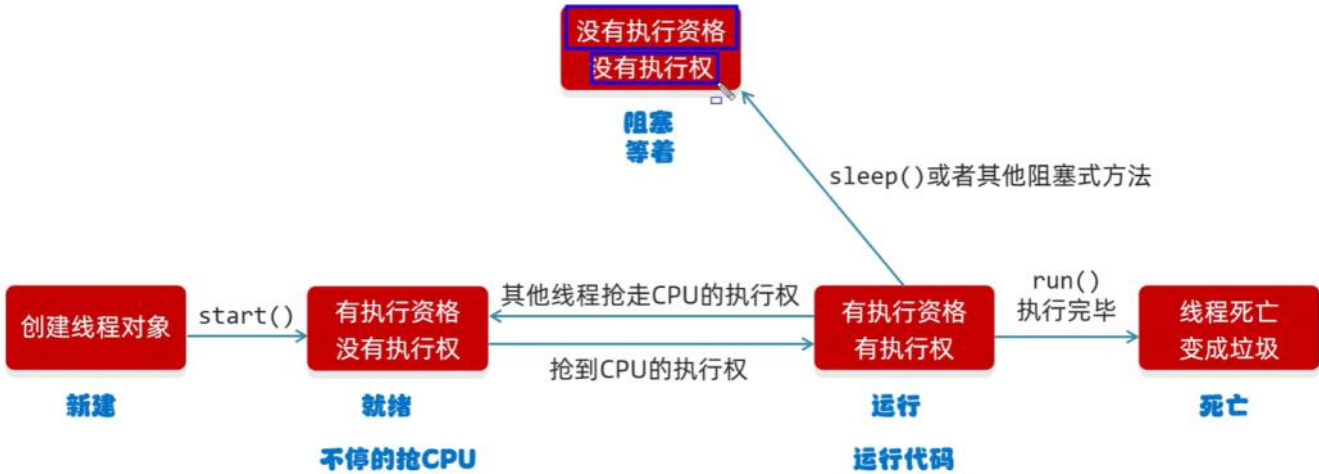
7、守护线程会在非守护线程结束之后陆续结束，可能跑完或跑不完

```
t1.setDaemon(true);
```

8、出让线程让出现次数相对均匀

9、插入线程让一个先执行完之后再执行另一个

线程的生命周期



同步代码块/方法

2026年9月22日 15:19

1、思路：

通过static让票数共享（第二种实现方法就不必static），使用synchronized锁起代码块
锁对象obj需要是唯一的，可以使用类的字节码文件 Mythread.class

```
import java.util.Objects;

public class Mythread extends Thread{
    static int ticket=0;
    //    static Object obj=new Object();

    @Override
    public void run() {
        while(true){
            synchronized (Mythread.class){
                if(ticket<100){
                    try {
                        Thread.sleep(50);
                    } catch (InterruptedException e) {
                        throw new RuntimeException(e);
                    }
                    ticket++;
                    System.out.println(getName()+"卖第"+ticket+"张票");
                }
                else{
                    break;
                }
            }
        }
    }
}

public class WorkDemo {
    public static void main(String[] args) {
        Mythread mt1 = new Mythread();
        Mythread mt2 = new Mythread();
        Mythread mt3 = new Mythread();

        mt1.setName("1号");
        mt2.setName("2号");
        mt3.setName("3号");

        mt1.start();
    }
}
```

```

        mt2.start();
        mt3.start();
    }
}

```

2、同步方法

步骤：选中循环之内的部分按ctrl+alt+m可以将方法独立出来，再在方法上加上synchronized，删掉原来的，成为同步方法

特点：1、同步方法是锁住方法里面所有的代码

2、锁对象不能自己指定

·非静态：this

·静态：当前类的字节码文件对象

```

import java.util.Objects;

public class Mythread extends Thread{
    static int ticket=0;
    @Override
    public void run() {
        while(true){
            if (method()) break;
        }
    }

    static private synchronized boolean method() {
        if(ticket<100){
            try {
                Thread.sleep(50);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
            ticket++;
            System.out.println(Thread.currentThread().getName()+"卖 第"+ticket+"张票");
        }
        else{
            return true;
        }
        return false;
    }
}

```

lock锁

2026年9月22日 17:13

1、

Lock是接口不能直接实例化，这里采用实现类ReentrantLock来实例化
因为Lock锁在break时会异常跳出，所以在finally处放置unlock一定执行
步骤：选中循环内的代码块，按ctrl+alt+t选择trycatchfinally自动调出

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class Mythread extends Thread{
    static int ticket=0;
    static Lock lock=new ReentrantLock();
    @Override
    public void run() {
        while(true){
            lock.lock();
            try {
                if(ticket<100){
                    Thread.sleep(50);
                    ticket++;
                    System.out.println(getName()+"卖第"+ticket+"张票");
                }
                else{
                    break;
                }
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            } finally {
                lock.unlock();
            }
        }
    }
}
```

线程池

2026年9月26日 15:28

1、Executors：线程池的工具类通过调用方法返回不同类型的线程池对象

<code>public static ExecutorService newCachedThreadPool()</code>	创建一个没有上限的线程池
<code>public static ExecutorService newFixedThreadPool(int nThread)</code>	创建有上限的线程池

2、主要核心原理

- 1、创建一个池子，池子中是空的
- 2、提交任务时，池子会创建新的线程对象，完毕后，线程归还给池子，下次再提交任务时，直接复用已有的线程
- 3、如果提交任务时没有空闲线程，也没法创建新的线程，任务就会排队等待

3、代码实现

创建三个线程，最后关闭线程池

```
ExecutorService pool1 = Executors.newCachedThreadPool();
pool1.submit(new MyThread());
pool1.submit(new MyThread());
pool1.submit(new MyThread());

pool1.shutdown();
```

创建上限为4的线程

```
ExecutorService pool1 = Executors.newFixedThreadPool(4);
pool1.submit(new MyThread());
pool1.submit(new MyThread());
pool1.submit(new MyThread());
pool1.submit(new MyThread());
pool1.submit(new MyThread());
pool1.submit(new MyThread());
```

4、自定义线程池元素

- 1、核心线程数量
- 2、线程池中最大线程的数量
- 3、空闲时间（值）
- 4、空闲时间（单位） 使用TimeUnit指定
- 5、阻塞队列
- 6、创建线程的方式

7、要执行的任务过多时的解决方案

```
ThreadPoolExecutor pool2 = new ThreadPoolExecutor(  
    3,  
    6,  
    60,  
    TimeUnit.SECONDS,  
    new ArrayBlockingQueue<>(3),  
    Executors.defaultThreadFactory(),  
    new ThreadPoolExecutor.AbortPolicy()  
);
```

5、自定义线程池排队机制

先一部分核心线程，再一部分排队等待核心线程，再一部分临时线程，最后多的去掉
任务拒绝策略：ThreadPoolExecutor.AbortPolicy，默认，丢弃任务并抛异常

6、线程池多大合适

16核24线程

1、CPU密集型运算： 最大并行数+1

2、I/O密集型运算： 最大并行数*期待CPU利用率*总时间/CPU计算时间

方法引用

2026年9月26日 16:35

1、方法引用条件

- 1、引用处需要是函数式接口@FunctionalInterface
- 2、被引用的方法需要已经存在
- 3、被引用方法的形参和返回值需要跟抽象方法的保持一致
- 4、被引用方法的功能需要满足当下的要求

2、实例

```
Integer[] arr = {1, 5, 10, 2, 4};
Arrays.sort(arr, new Comparator<Integer>() {
    @Override
    public int compare(Integer o1, Integer o2) {
        return o2 - o1;
    }
});
Arrays.stream(arr).forEach(System.out::println);
```

另外：lambda简化写法

```
Arrays.sort(arr, (o1, o2) -> o2 - o1);
```

对于其中的Comparator方法ctrl+b跟进得到@FunctionalInterface, 可以用具有相同的另一个方法替换, 实现一样功能

```
public static int sub(Integer a, Integer b) {
    return b - a;
}
Arrays.sort(arr, Demo::sub);
```

3、方法引用符 ::

方法引用分类

2026年9月26日 16:48

1、引用静态方法

```
ArrayList<String> list = new ArrayList<>();  
Collections.addAll(list, "1", "2", "3");  
list.stream().map(Integer::parseInt).forEach(System.out::println);
```

2、引用成员方法

- 1、引用其他类的成员方法
- 2、引用本类的成员方法
- 3、引用父类的成员方法

3、引用构造方法

0、其他调用方法

- 1、使用类名引用成员方法
- 2、引用数组的构造方法

File的三种构造方法

2026年9月26日 17:08

- 1、File对象表示路径，可以是存在的或是不存在的
- 2、绝对路径是带盘符的
相对路径是不带盘符的，默认到当前项目下去找
- 3、构造方法
 - 1、public File (String pathname
 - 2、public File (String parent , String child)
 - 3、public File (File parent ,String child)
- 4、\代表转义字符，\\代表反斜线
所以复制路径过来的时候会自动变成\\，
使用第二种构造方法是可以避免linux中的符号为//

File常见的成员方法

2026年9月26日 17:08

1、判断、获取

```
File f1 = new File("C:\\Users\\ttt\\Desktop\\a.txt");
```

方法名称	说明
public boolean isDirectory()	判断此路径名表示的File是否为文件夹
public boolean isFile()	判断此路径名表示的File是否为文件
public boolean exists()	判断此路径名表示的File是否存在
public long length()	返回文件的大小（字节数量）
public String getAbsolutePath()	返回文件的绝对路径
public String getPath()	返回定义文件时使用的路径
public String getName()	返回文件的名称，带后缀
public long lastModified()	返回文件的最后修改时间（时间毫秒值）

2、创建删除

方法名称	说明
public boolean createNewFile()	创建一个新的空的文件
public boolean mkdir()	创建单级文件夹
public boolean mkdirs()	创建多级文件夹
public boolean delete()	删除文件、空文件夹

3、获取并遍历

方法名称	说明
public File[] listFiles()	获取当前该路径下所有内容

- 1、当路径不存在时，返回null
- 2、当路径是文件时，返回null
- 3、当路径是空文件夹时，返回一个长度为0的数组

- 4、当路径时一个有内容的文件夹时，将里面所有文件和文件夹的路径放在数组返回
- 5、当路径时需要权限才能访问的文件夹时，返回null

4、

File示例方法

2026年9月26日 18:50

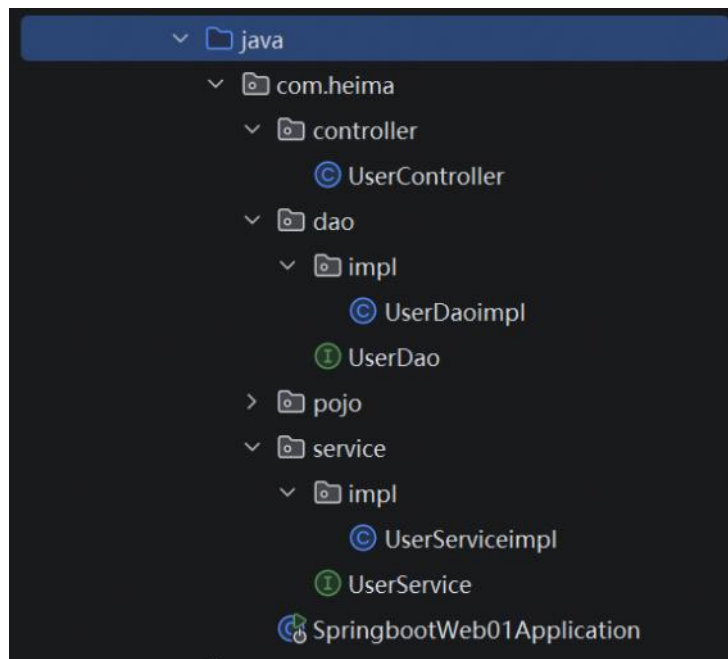
1、查找指定路径里面是否存在指定结尾的文件

```
public static boolean HaveAvi(File file) {  
    File[] f = file.listFiles();  
    for (File f1 : f) {  
        if(f1.isFile() && f1.getName().endsWith("avi")) {  
            return true;  
        }  
    }  
    return false;  
}
```

三层架构

2026年9月29日 10:11

- 1、Controller：控制层，接收前端发送的数据，对请求进行处理，并响应数据
- 2、Service：业务逻辑层，处理具体的业务逻辑
- 3、Dao：数据访问层，负责数据访问操作，包括数据的增删改查



impl指实现类

先是controller接收到请求，其中需要service获取响应的数据
而service进行逻辑处理需要dao获取原始数据
最后依次返回

